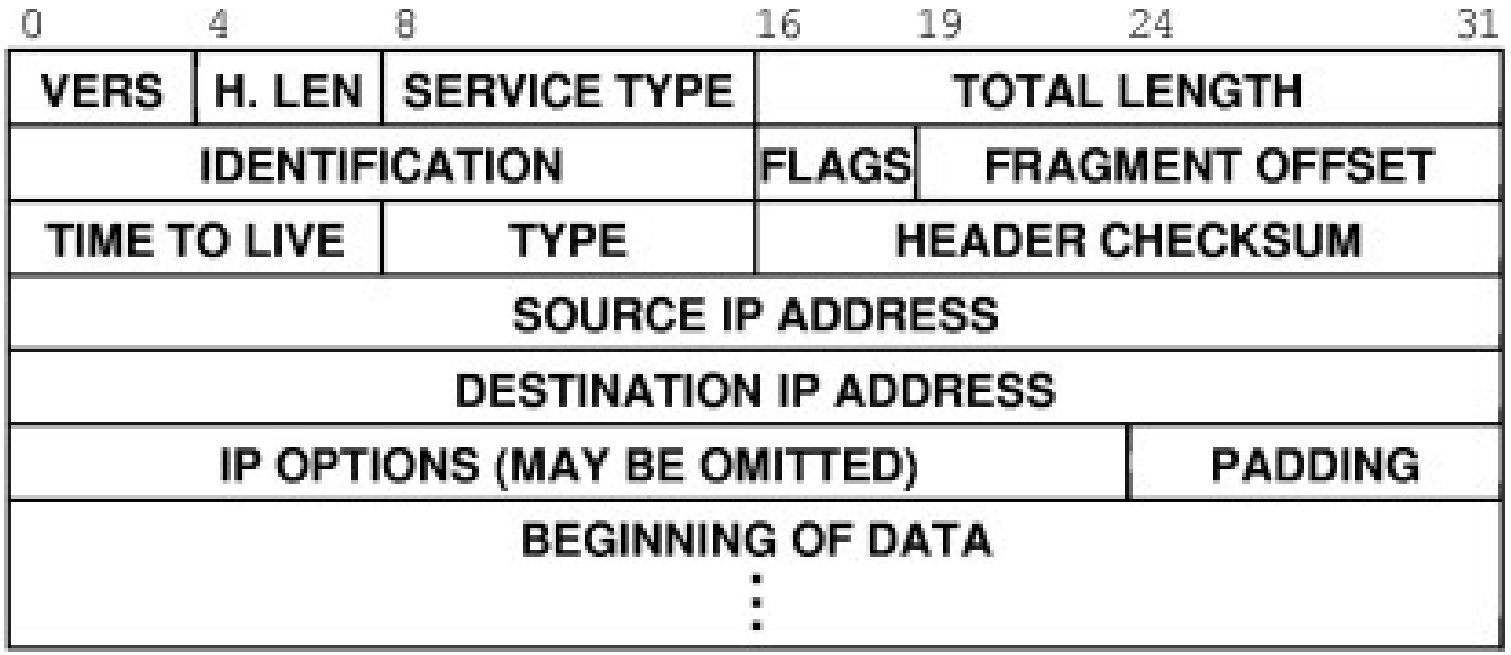
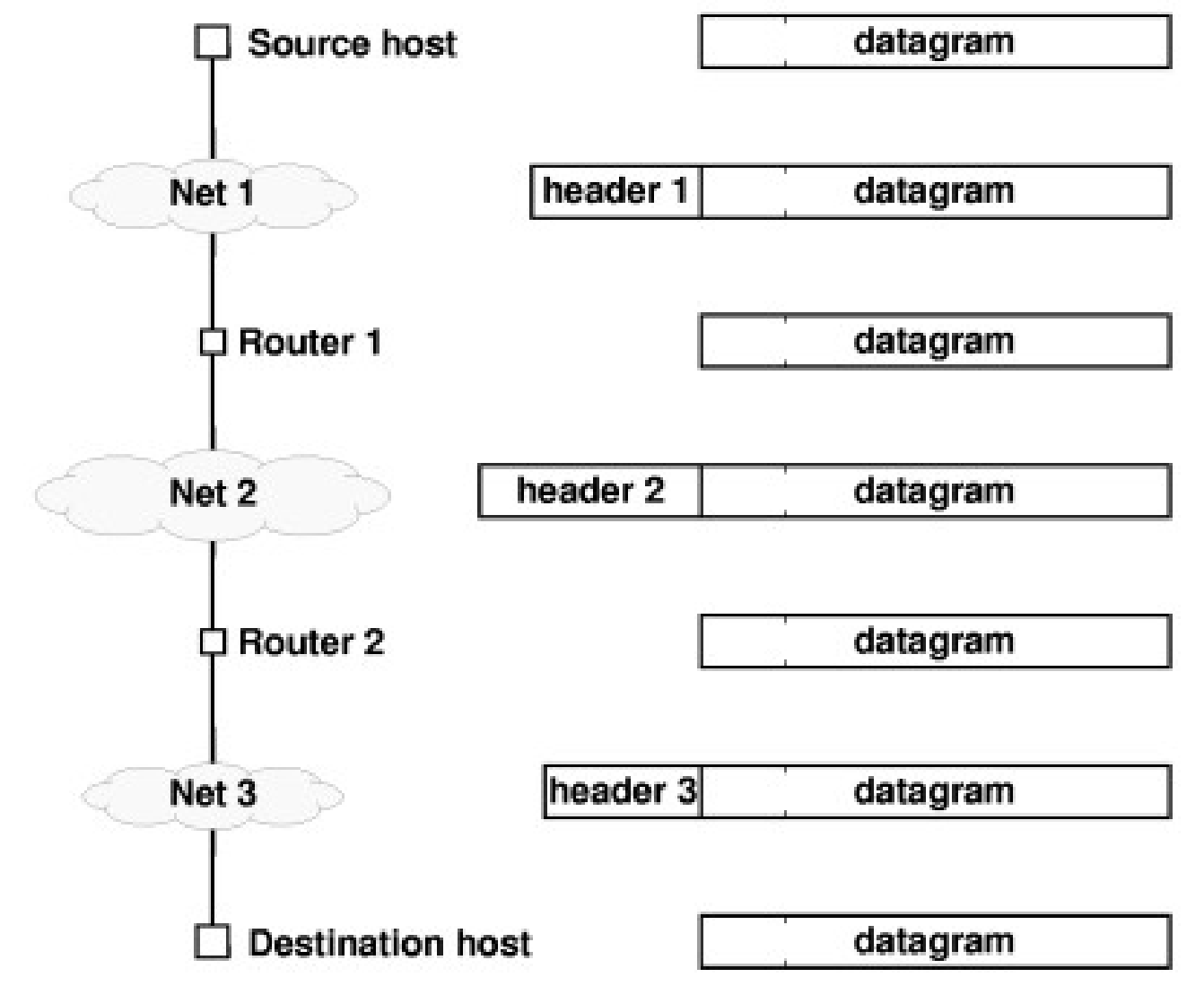
The IP Datagram Header Format



Encapsulation at work



Forwarding IP Datagrams

- Recall the use of **encapsulation** when forwarding datagrams across a LAN:
  - Datagrams are placed inside the Data field of a frame which is associated with the LAN.
- However, the process is hampered by a restriction imposed by each LAN technology:
  - Each LAN technology can only carry a **maximum** amount of data within its frame.

Maximum Transmission Unit (MTU)

- This restriction is known as the ***Maximum Transmission Unit (MTU)***.
- Examples include:
  - Token Ring (16Mbps) MTU = 17,914 bytes,
  - Ethernet MTU = 1,500 bytes.
- Hence, <u>each</u> datagram (header <u>and</u> data field) must be smaller or equal to the network MTU.

Maximum Transmission Unit (MTU)

- In an *internetwork* which is comprised of many interconnected LANs there are potentially many MTU restrictions in place:

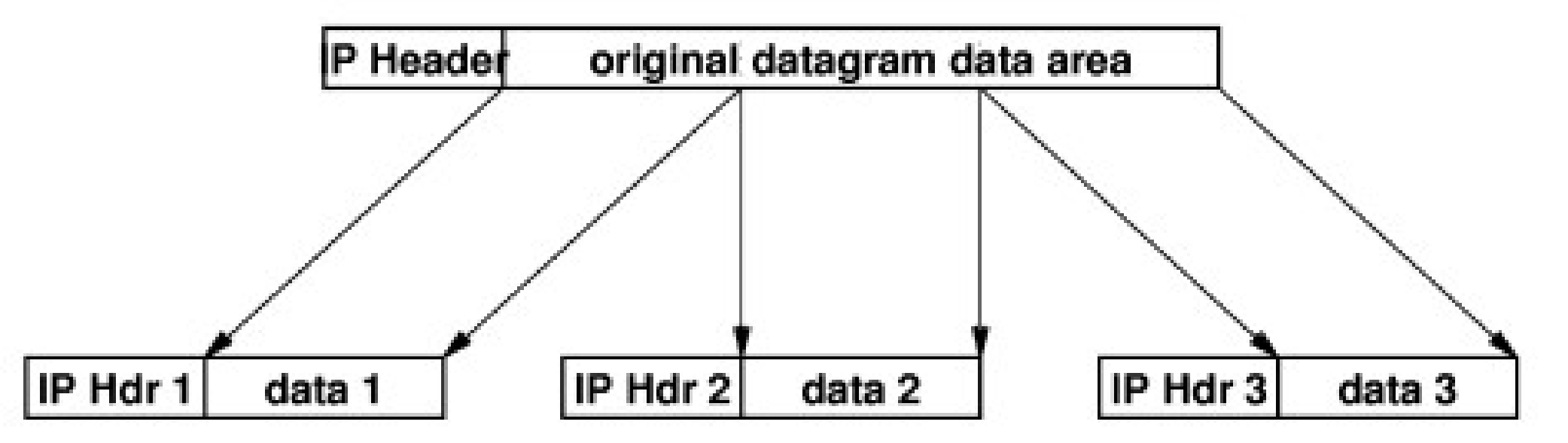
Maximum Transmission Unit (MTU)

- Datagrams that need to traverse a network that has a MTU value smaller than the size of the datagram need to be dealt with.
- To resolve this problem IP routers use a technique known as **Fragmentation***:*
  - This technique involves *splitting* or *fragmenting* the Datagram into smaller ‘pieces’.

IP Fragmentation

- Typically *routers* make this decision.
- The router uses *the network MTU* <u>and</u> *the datagram header size* to calculate:
  - The maximum amount of data that can be sent within each fragment.
  - The number of fragments that will be required.
  - The next slide shows an overview of the process.

MTU and *Fragmentation* at work



IP Fragmentation Process

- From the example shown in the diagram three fragments are to be created.
- The process involves:
  - Creating copies of the original datagram header.
  - Each copy becomes the header for a fragment.
  - The Data field of the original datagram is divided with each division going to a separate fragment.

IP Fragmentation Process

- Changes are then made to some of the fields in the header section of each of the fragments.
- In particular:
  - The **FLAGS** field.
  - The **FRAGMENT OFFSET** field.
  - The **IDENTIFICATION** field.
- It is important to note that each fragment contains the same *source* <u>and</u> *destination IP address* as the original datagram.

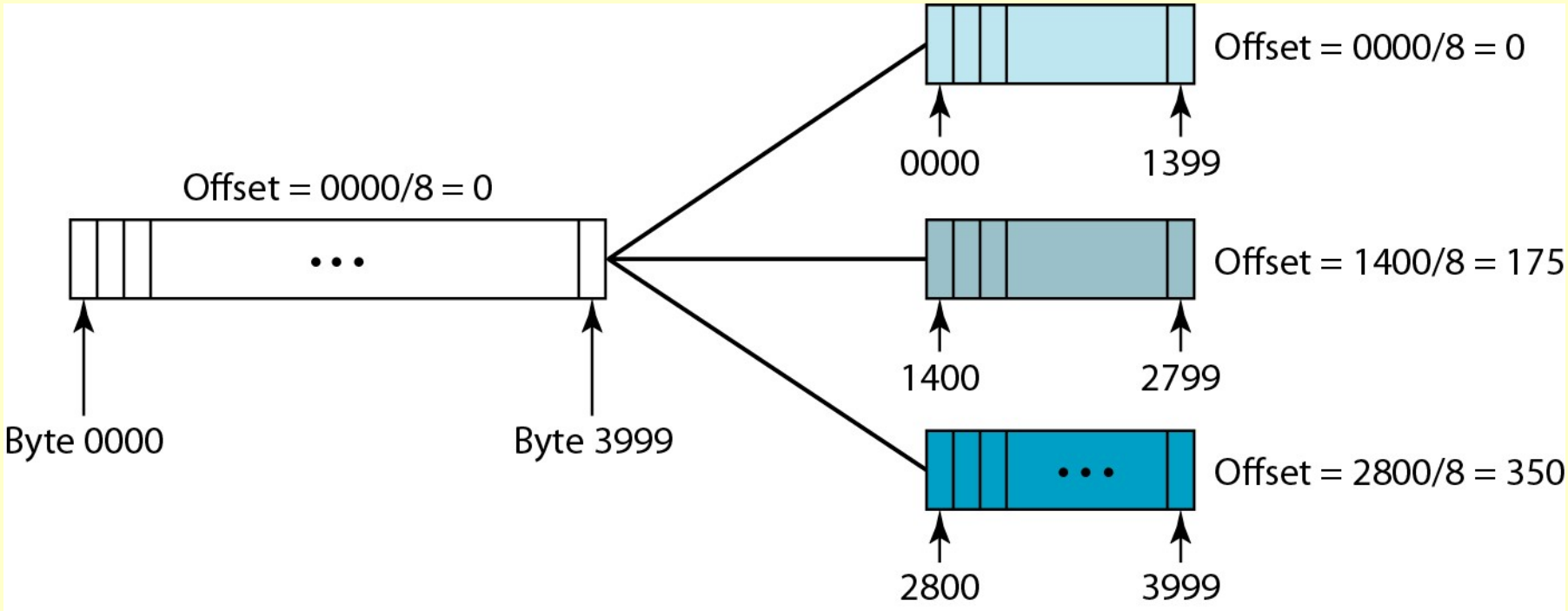
IP Fragmentation Process

- The **FLAGS** field:
  - This is used to indicate if a datagram is allowed to be fragmented,
  - If allowed, this field also indicates if this fragment is the last fragement (1 = more fragments to come, 0 = this is the last).
- The **IDENTIFICATION** field.
  - This is a unique identifier for the original datagram which is used for re-assembly.

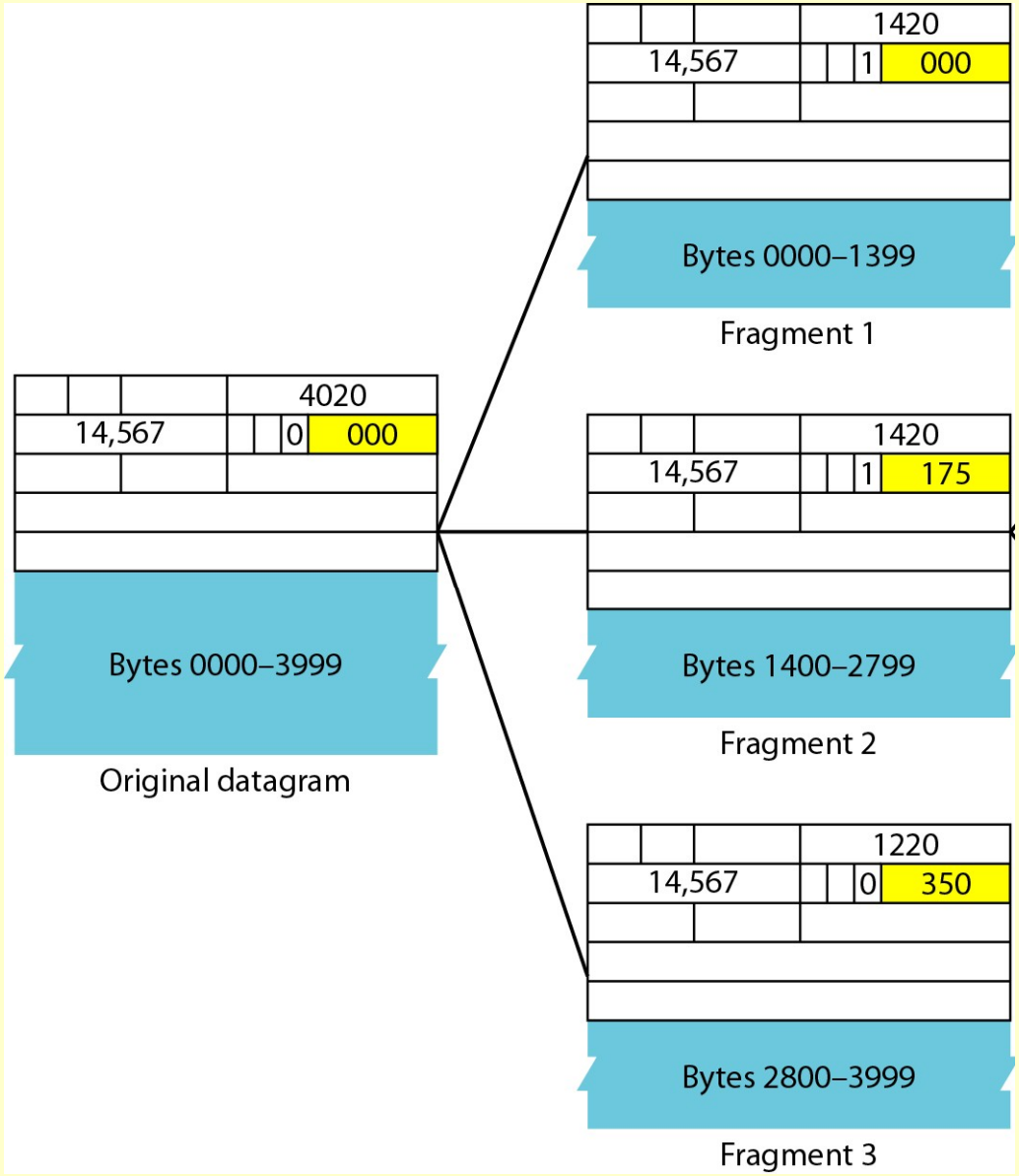
IP Fragmentation Process

- The **FRAGMENT OFFSET** field:
  - This field is used to determine when each of the "data splits" belongs in relation to the data field of the original datagram.
  - It is measured in units of 8 bytes.
  - The following example explains the concepts.

IP Fragmentation Example



IP Fragmentation Example



IP Fragmentation - Reassembly

- The *Destination host* is responsible for the reassembly of the fragments.
- This is appropriate for two reasons:
  - IP does not necessarily route all datagrams belonging to a single message along the same path,
  - The routes taken by each datagram can change dynamically depending on network conditions.

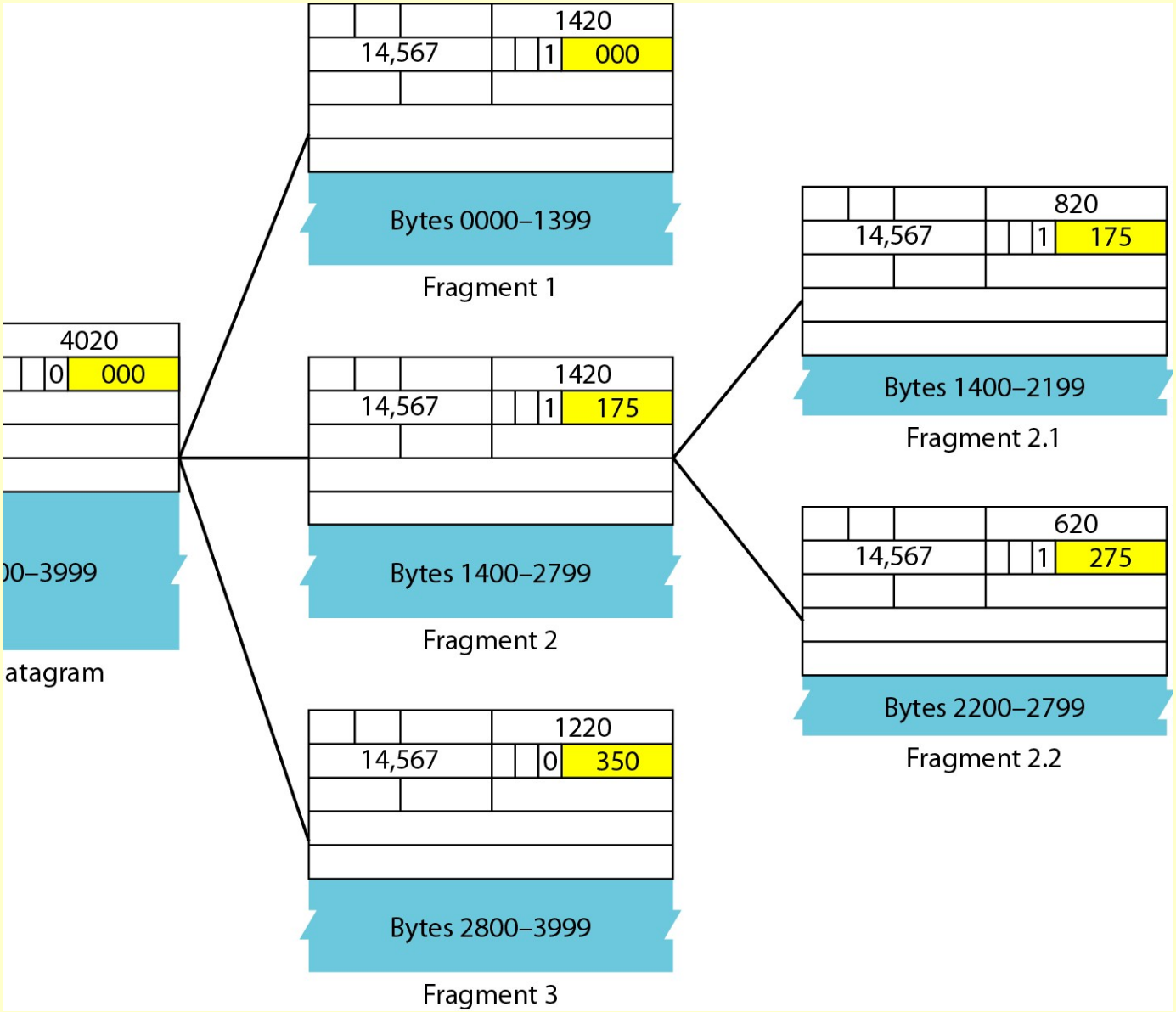
IP Fragmentation - Reassembly

- The destination host checks the ***Flags*** field to determine if the datagram is a fragment and if so, whether it is the last fragment.
- The ***Identification*** field is used to uniquely identify fragments belonging to a single datagram.
- The ***Offset*** field is used to reconstruct the original datagram *data field*.

IP Fragmentation - Reassembly

- In the event of a *lost fragment* the destination host saves the fragments for a <u>fixed</u> period of time only
- In essence, the reassembly process is *all-or-nothing.*

Fragmenting a Fragment?

- When a *fragment* arrives at a network that has a smaller *MTU,* it can be further fragmented.
- The *destination host* does not know whether a fragment is derived directly from a *datagram* or from another *fragment*

Fragmenting a Fragment Example